



Imersão GenAI para Desenvolvedores

Módulo 2

Preâmbulo: Neste módulo, exploraremos algumas técnicas de Engenharia de Prompt.

Versão: 1.1

Sumário

I	Conceitos fundamentais sobre Large Language Models (LLMs)	2
II	Exercício 1: Escolha seu assistente de AI	4
III	Exercício 2: First Prompt	6
IV	Exercício 3: Few Shot Prompting	8
V	Exercício 4: Tags XML	9
VI	Exercício 5: Role Playing	10
VII	Exercício 6: Structured Output	11
VIII	Exercício 7: Prompt Chaining	12
IX	Entrega e Avaliação entre Pares	14

Capítulo I

Conceitos fundamentais sobre Large Language Models (LLMs)

1. O que são LLMs?

- LLMs são modelos linguísticos treinados em grandes quantidades de texto, capazes de entender, gerar e processar linguagem natural de maneira semelhante aos humanos.
- Eles aprendem padrões e relações no texto, permitindo-lhes realizar uma ampla gama de tarefas, desde completar frases até responder perguntas e gerar textos coerentes.
- Saiba mais: [O que são grandes modelos linguísticos?](#)

2. Parâmetros (weights)

- Os parâmetros são os valores aprendidos que determinam o comportamento e as capacidades do modelo.
- O número de parâmetros, geralmente na ordem de bilhões (B) ou trilhões (T), é uma medida da capacidade e complexidade do modelo.
- Modelos maiores com mais parâmetros geralmente são capazes de desempenho melhor e mais nuances, mas também são mais desafiadores de treinar e servir.

3. Tokens e tokenização

- Antes de ser processado por um *Large Language Model* (LLM), o texto é dividido em tokens, que podem ser palavras, subpalavras ou até mesmo caracteres individuais.
- A tokenização é o processo de converter texto em uma sequência de tokens, que serve como entrada para o modelo.
- A contagem de tokens é uma medida importante, pois a maioria dos LLMs tem um comprimento máximo de sequência (por exemplo, 8192 tokens) e os custos são geralmente calculados por token.

- Saiba mais: [O que são tokens e como contá-los?](#)

4. Engenharia de Prompt

- A engenharia de prompt é a prática de projetar e refinar prompts para obter saídas desejadas de LLMs.
- Envolve a criação de instruções claras, fornecimento de contexto relevante e experimentação com diferentes formatos e estruturas de prompt.
- Técnicas comuns incluem *few-shot learning*, *chain-of-thought prompting* e o uso de separadores como tags XML ou estruturas JSON.

Capítulo II

	Exercício : 01
Escolha seu assistente de AI	
Pasta de entrega : <i>ex01/</i>	
Arquivos para entregar : provider.txt	
Funções ou bibliotecas autorizadas : Sem restrições	

Neste exercício você vai usar a API do provedor de AI de sua preferência.
Alguns provedores de AI onde você pode obter uma chave de API com 'free tier':

- [Google AI Studio](#)
- [Groq](#)
- [Open AI](#)

Escreva no documento **provider.txt**:

1. O passo a passo para obter uma chave de api.
2. Alguns modelos de LLM suportados pelo provedor, e seus **rate limits**.
3. A url para a documentação da API deste provedor, onde demonstre como utilizá-la para enviar um prompt.



Fique à vontade para adicionar informações sobre mais de um provider.



Não é necessário adicionar informações de pagamento. Utilize um provedor de AI que possua um limite de uso gratuito (free-tier). Não copie sua chave de api para o arquivo 'provider.txt'. Informe apenas o passo a passo de como obtê-la.

Capítulo III

	Exercício : 02
First Prompt	
Pasta de entrega : <i>ex02/</i>	
Arquivos para entregar : prompt.py	
Funções ou bibliotecas autorizadas : Sem restrições	

Neste exercício, você vai usar a API do Assistente de IA de sua preferência. Para interagir com esta API, você pode utilizar bibliotecas de requisições HTTP (requests, httpx) ou a SDK fornecida diretamente pelo seu provedor de LLM escolhido, como:

- [OpenAI Python Library](#)
- [Groq Python Library](#)
- [Google Generative AI SDK](#).

Você pode, também, instalar o [ollama](#) localmente. Deixamos na intranet um `docker-compose.yml` preparado para o uso local com o modelo `gemma3:1b`. Neste caso, você pode utilizar a SDK `ollama` disponível no PyPI: [<https://pypi.org/project/ollama/>].

1. Construa um **programa** que receba dois argumentos:
 - um prompt
 - uma temperatura (argumento opcional. Se não enviado, defina uma temperatura padrão.)
2. Seu programa deve imprimir a resposta gerada pela IA, como no exemplo abaixo:

```
?> python3 prompt.py "Explique o que é Python em uma frase." 1.0
Python é uma linguagem de programação de alto nível, interpretada e de propósito geral, conhecida por sua sintaxe clara e legibilidade.
```



Chaves de API devem ser definidas em um arquivo `'.env'` ou diretamente nas variáveis de ambiente. Não defina chaves de API em seu próprio código.



Como diferentes temperaturas se comportam com o modelo escolhido?



Escolha um model o que permite uma quantidade razoável de requisições para o limite de uso gratuito. Fique à vontade para experimentar e alternar entre diferentes provedores de IA e modelos ao longo deste curso.

Capítulo IV

	Exercício : 03
Few Shot Prompting	
Pasta de entrega : <i>ex03/</i>	
Arquivos para entregar : fewshot.py	
Funções ou bibliotecas autorizadas : Sem restrições	

1. Crie um programa que traduza termos técnicos de informática, utilizando a técnica de [few shot prompting](#).
2. Seu programa deverá enviar ao menos três exemplos de entrada e saída esperada, e seu comportamento deve ser conforme abaixo:

```
?> python3 fewshot.py SQL
linguagem de consulta estruturada
```

Capítulo V

	Exercício : 04
Tags XML	
Pasta de entrega : <i>ex04/</i>	
Arquivos para entregar : xml.py	
Funções ou bibliotecas autorizadas : Sem restrições	

Quando seus prompts envolvem vários componentes como contexto, instruções e exemplos, o uso de separadores pode fazer toda a diferença. Separadores, como tags XML ou estruturas JSON, ajudam qualquer LLM a analisar seus prompts com mais precisão, resultando em saídas de maior qualidade.

1. Crie um programa que resuma um texto em no máximo 20 palavras.
2. Seu prompt deve organizar, em [tags XML](#), as partes:
 - `<instrucoes>`: descrevendo como o modelo deve agir.
 - `<exemplos>`: incluindo pelo menos uma entrada e saída esperada.
 - `<texto>`: onde será colocada a entrada do usuário.
3. Exemplo de comportamento esperado:

```
?> python3 xml.py "Durante a Idade Média, as universidades europeias surgiram como centros de conhecimento, reunindo estudiosos de diversas áreas. Elas desempenharam papel essencial na preservação e transmissão de saberes clássicos, além de estimular debates filosóficos e científicos que moldaram o pensamento ocidental e prepararam terreno para o Renascimento."
Universidades medievais preservaram saber clássico, promoveram debates e criaram base intelectual que impulsionou o Renascimento europeu.
```

Capítulo VI

	Exercício : 05
Role Playing	
Pasta de entrega : <i>ex05/</i>	
Arquivos para entregar : roleplay.py	
Funções ou bibliotecas autorizadas : Sem restrições	

O papel de sistema define as regras e o comportamento-base do modelo, funcionando como instruções de mais alta prioridade em uma conversa.

1. Crie um programa `roleplay.py` que simule um consultor de viagens especializado.
2. O programa deve receber um destino como argumento.
3. Implemente um prompt de **sistema** que instrua o modelo a assumir o papel de um consultor de viagens especializado no destino solicitado e que retorne as 5 melhores atrações relativas àquele destino.

Exemplo de saída:

```
?> python3 roleplay.py "Paris"
1. Torre Eiffel
2. Museu do Louvre
3. Catedral de Notre-Dame
4. Arco do Triunfo
5. Basílica de Sacré-Cœur
```



O papel de sistema pode ser passado ao modelo de formas diferentes:

- Em modelos chat-based modernos, aparece como uma mensagem com role "system", antecedendo as mensagens do usuário e do assistant.
- Em APIs que usam prompts diretos, é incluído no início do prompt como instrução fixa;
- Em SDKs específicos, pode ser definido por parâmetros como `system_prompt`, `instruction` ou `persona`, sempre com prioridade sobre as mensagens do usuário para guiar o comportamento-base do modelo.

Capítulo VII

	Exercício : 06
Structured Output	
Pasta de entrega : <i>ex06/</i>	
Arquivos para entregar : structured.py	
Funções ou bibliotecas autorizadas : Sem restrições	

Os modelos podem ser orientados a fornecer saídas em [formatos específicos](#), como JSON ou XML. Isso é especialmente útil para integração com outros sistemas ou para garantir que a saída seja facilmente analisável.

1. Crie um programa que extraia informações de um texto e as retorne em formato JSON.
2. O programa deve receber um texto sobre uma pessoa como argumento.
3. Implemente um prompt que solicite ao modelo para extrair nome, idade, profissão e cidade, formatando a saída como JSON. Exemplo de saída:

```
?> python3 formatted.py "João Silva, 35 anos, é engenheiro e mora em São Paulo"
{"nome": "João Silva", "idade": 35, "profissão": "engenheiro", "cidade": "São Paulo"}
```



Algumas SDKs aceitam que um modelo 'pydantic' seja enviado como base para uma saída estruturada. A SDK escolhida possui esta funcionalidade?

Capítulo VIII

	Exercício : 07
Prompt Chaining	
Pasta de entrega : <i>ex07/</i>	
Arquivos para entregar : chaining.py	
Funções ou bibliotecas autorizadas : Sem restrições	

O [encadeamento de prompts](#) é uma técnica de Engenharia de Prompt que consiste em quebrar uma tarefa complexa em etapas sequenciais, onde a saída de um prompt serve como entrada para o próximo.

1. Crie um programa `chaining.py` que implemente uma cadeia de prompts (prompt chaining) para aprimorar uma descrição de um produto.
2. O programa deve receber uma descrição breve de produto como argumento.
3. Implemente três passos sequenciais:
 - Primeiro prompt: Solicite ao modelo que gere uma descrição mais detalhada e persuasiva do produto.
 - Segundo prompt: Use a saída do primeiro prompt para solicitar ao modelo que converta a descrição em um anúncio publicitário curto e criativo.
 - Terceiro prompt: Solicite a tradução para o inglês.

```
?> python3 chaining.py "fone de ouvido sem fio"
[Descrição detalhada gerada pelo primeiro prompt]
---
[Anúncio publicitário gerado pelo segundo prompt]
---
[Tradução para a língua inglesa]
```



Utilize a saída do primeiro prompt como entrada para o segundo, mantendo o contexto coerente.

Capítulo IX

Entrega e Avaliação entre Pares

- Entregue seu projeto em seu repositório *Git* disponível na página do projeto na intranet.
- Apenas o trabalho dentro do seu repositório será avaliado durante a defesa. Não hesite em verificar os nomes de seus arquivos e pastas para garantir que estejam corretos.
- No horário da avaliação, o avaliador se dirigirá à estação de trabalho do aluno a ser avaliado para realizar os testes. Um clone do repositório deverá ser realizado em uma nova pasta, e estes são os arquivos que serão avaliados.